ELEC 475 Lab 2

Group 1 Lab 2

Geoffrey Lehman 20331085

Oskar Petro Lomheim 20328881

GitHub Repo: https://github.com/geoffrey329/ELEC_475_Lab2

Step 1 (architectural specs)

This section specifies the four architectures compared in this lab. Every model receives a normalized RGB tensor of shape 3×64×64 and outputs K logits, where K = 10 for CIFAR-10 and K = 200 for Tiny ImageNet-200. All four were trained with the same engine (train.py): cross-entropy loss, Adam (learning rate 10^{-3}, weight decay 10^{-4}) and batch size 128. Layer dimensions, receptive fields and parameter counts were derived by hand with the formulas below and then checked against the torchinfo summaries and ModelProfiler output; every hand-computed parameter total matches the tool output exactly.

Notation and Formulas Used

2-D convolution. Output channel c_o at spatial position (i, j), for a layer with C_{in} input channels, kernel size K, stride S and padding P:

$$Y[c_o, i, j] = b[c_o] + \Sigma_{c=1}^{C_{in}} \Sigma_{u=0}^{K-1} \Sigma_{v=0}^{K-1} W[c_o, c, u, v] \cdot X[c, S \cdot i + u - P, S \cdot j + v - P]$$

Spatial output size (convolution and pooling):

$$N_{out} = \lfloor (N_{in} - K + 2P) / S \rfloor + 1$$

Receptive field (RF). r_l is the side length, in input pixels, of the input region that one unit of layer l can see. j_l (the "jump") is the distance in input pixels between two adjacent units of layer l. Both start at $r_0 = j_0 = 1$:

$$r_l = r_{l-1} + (K_l - 1) \cdot j_{l-1}, \qquad j_l = j_{l-1} \cdot S_l$$

Activations, normalization layers, dropout and flattening leave r and j unchanged. The RF values are theoretical: when r > 64, part of the region falls in the zero-padding.

Parameter and compute counts:

- Convolution: $C_{out} \cdot (C_{in} \cdot K^2 + 1)$ parameters and $C_{out} \cdot H_{out} \cdot W_{out} \cdot C_{in} \cdot K^2$ MACs.
- Fully connected (Linear): $N_{out} \cdot (N_{in} + 1)$ parameters and $N_{in} \cdot N_{out}$ MACs.
- BatchNorm2d: 2C trainable parameters (γ, β), plus 2C running-statistic buffers that are not trained.
- Pooling, Tanh/ReLU, LRN, Dropout and Flatten: no parameters.

- One multiply-accumulate (MAC) ≈ 2 FLOPs. The MAC columns in the tables below count only convolution and linear layers, which is why they are slightly lower than the profiler totals: the profiler also counts pooling and normalization operations.

Classifier output. The logits $z \in \mathbb{R}^K$ are converted to class probabilities with a softmax, and the network is trained to minimize the cross-entropy against the true label y:

$$p_k = e^{z_{\{k\}}} / \Sigma_{m=1}^{K} e^{z_{\{m\}}}, \qquad L = -\log p_y$$

3.1 Base LeNet-5 (models/lenet.py)

Textual Description

Base LeNet-5 follows the classical design of LeCun et al. (1998). Two changes adapt it to this lab: the first convolution takes 3 RGB channels instead of 1 grayscale channel, and the input is 64×64 instead of 32×32. The feature extractor has two identical stages, each made of a 5×5 "valid" convolution (P = 0, S = 1), a Tanh activation and non-overlapping 2×2 average pooling (S = 2). Stage 1 uses 6 filters and stage 2 uses 16. The resulting 16×13×13 feature volume is flattened into a 2,704-dimensional vector. That vector passes through three fully connected layers (2,704 → 120 → 84 → K), with Tanh after the first two. The last layer outputs raw logits, and the softmax is applied inside the cross-entropy loss.

Mathematical Description

Spatial dimensions, using $N_{out} = \lfloor (N_{in} - K + 2P)/S \rfloor + 1$:

$$\text{Conv1: } (64 - 5 + 0)/1 + 1 = 60 \;\rightarrow\; \text{Pool1: } (60 - 2)/2 + 1 = 30 \;\rightarrow\; \text{Conv2: } (30 - 5)/1 + 1 = 26$$
$$\rightarrow\; \text{Pool2: } (26 - 2)/2 + 1 = 13$$
$$\text{Flatten: } 16 \times 13 \times 13 = 2{,}704$$

Activation function and its derivative:

$$\tanh(z) = (e^{z} - e^{-z}) / (e^{z} + e^{-z}), \qquad d/dz \tanh(z) = 1 - \tanh^2(z) \in (0, 1]$$

The derivative is at most 1 and approaches 0 when |z| is large. These saturating gradients shrink as they are multiplied back through successive layers. Average pooling is a fixed linear low-pass filter: it averages each non-overlapping window and passes 1/4 of the incoming gradient to each of the four inputs.

$$Y[c, i, j] = \tfrac{1}{4} \Sigma_{u=0}^{1} \Sigma_{v=0}^{1} X[c, 2i + u, 2j + v]$$

The fully connected layers compute $h_1 = \tanh(W_1 x + b_1)$, $h_2 = \tanh(W_2 h_1 + b_2)$ and $z = W_3 h_2 + b_3$.

Layer Dimensions, Receptive Field and Parameter Breakdown

Table 1.1: Base LeNet-5 layer specification (input 3×64×64).

Layer	Operation	Output (C×H×W)	RF (px)	Jump	Params	MACs
Conv1	Conv2d(3→6, 5×5, s=1, p=0) → Tanh	6×60×60	5	1	456	1,620,000
Pool1	AvgPool2d(2×2, s=2)	6×30×30	6	2	0	—
Conv2	Conv2d(6→16, 5×5, s=1, p=0) → Tanh	16×26×26	14	2	2,416	1,622,400

Pool2	AvgPool2d(2×2, s=2)	16×13×13	16	4	0	—
Flatten	16×13×13 → 2,704	2,704	—	—	0	—
FC1	Linear(2,704→120) + Tanh	120	—	—	324,600	324,480
FC2	Linear(120→84) + Tanh	84	—	—	10,164	10,080
FC3 (out)	Linear(84→K)	K (10 / 200)	—	—	850	840
Total	**K = 10 (CIFAR-10)**				**338,486**	**3,577,800**

Receptive field: r = 5 → 6 → 14 → 16 px, with a final jump of 4. Each of the 13×13 units in the last feature map therefore sees only a 16×16 patch, about 6% of the 64×64 image. The convolutional layers capture local edges and textures only; any reasoning about the whole object has to happen in the fully connected layers.

Parameter breakdown:

- Convolutional layers: 456 + 2,416 = 2,872 parameters (0.85%).
- Fully connected layers: 324,600 + 10,164 + 850 = 335,614 parameters (99.15%). FC1 alone holds 324,600, or 95.9% of the model, because it connects every one of the 2,704 flattened features to 120 neurons.
- Total: **338,486** parameters for K = 10. For Tiny ImageNet (K = 200), FC3 becomes 84·200 + 200 = 17,000, giving **354,636**.
- Compute: the two convolutions dominate, with 1.62M MACs each out of 3.58M analytic MACs. The profiler reports 3.59M MACs and 7.17 MFLOPs.

3.2 Modernized LeNet (models/lenet_modern.py)

Textual Description

Modernized LeNet keeps LeNet-5's filter counts (6 and 16) and its 120 → 84 → K classifier, but changes three things, which are the In-Lab ablations:

- Every Tanh is replaced with ReLU (Ablation 1).
- The 2×2 average pools are replaced with overlapping 3×3 max pools at stride 2 (Ablation 2).
- Both 5×5 convolutions use "same" padding, P = (K − 1)/2 = 2, so convolution no longer shrinks the feature maps and only the pooling layers downsample. This keeps enough spatial size for the larger overlapping pooling windows.

The final feature volume is 16×15×15, which flattens into a 3,600-dimensional vector.

Mathematical Description

$$\text{Conv1: } (64-5+4)/1+1=64 \;\rightarrow\; \text{Pool1: } \lfloor (64-3)/2 \rfloor + 1 = \lfloor 30.5 \rfloor + 1 = 31 \;\rightarrow\; \text{Conv2: } (31-5+4)/1+1=31 \;\rightarrow\; \text{Pool2: } \lfloor (31-3)/2 \rfloor + 1 = 15$$

$$\text{Flatten: } 16 \times 15 \times 15 = 3{,}600$$

In Pool1, the floor operation means the 31 windows cover rows and columns 0–62, so the last row and column of the 64×64 map are dropped. In Pool2, 2·14 + 3 = 31 covers the map exactly.

$$\text{ReLU}(z) = \max(0, z), \qquad d/dz\ \text{ReLU}(z) = 1 \text{ for } z > 0,\ 0 \text{ for } z < 0$$

ReLU does not saturate for positive inputs: the gradient passes back with a gain of exactly 1, which avoids the gradient shrinkage of Tanh. It also produces sparse activations, because negative inputs are set to zero.

$$Y[c, i, j] = \max_{0 \le u, v \le 2} X[c, 2i + u, 2j + v]$$

Because S = 2 < K = 3, adjacent pooling windows overlap by K − S = 1 row or column. Max pooling keeps the strongest response in each window rather than blurring it, and routes the gradient only to the winning input.

Layer Dimensions, Receptive Field and Parameter Breakdown

Table 1.2: Modernized LeNet layer specification (input 3×64×64).

Layer	Operation	Output (C×H×W)	RF (px)	Jump	Params	MACs
Conv1	Conv2d(3→6, 5×5, s=1, p=2) → ReLU	6×64×64	5	1	456	1,843,200
Pool1	MaxPool2d(3×3, s=2)	6×31×31	7	2	0	—
Conv2	Conv2d(6→16, 5×5, s=1, p=2) → ReLU	16×31×31	15	2	2,416	2,306,400
Pool2	MaxPool2d(3×3, s=2)	16×15×15	19	4	0	—
Flatten	16×15×15 → 3,600	3,600	—	—	0	—
FC1	Linear(3,600→120) + ReLU	120	—	—	432,120	432,000
FC2	Linear(120→84) + ReLU	84	—	—	10,164	10,080
FC3 (out)	Linear(84→K)	K (10 / 200)	—	—	850	840
Total	**K = 10 (CIFAR-10)**				**446,006**	**4,592,520**

Receptive field: r = 5 → 7 → 15 → 19 px, with a final jump of 4. Each output unit sees a 19×19 patch, about 9% of the image. That is larger than LeNet-5's 16 px because each pooling window spans 3 pixels instead of 2.

Parameter breakdown:

- Convolutional parameters are identical to LeNet-5 (2,872). Padding, activation choice and pooling type have no learnable parameters.
- The larger flattened vector (3,600 vs 2,704, +33%) increases FC1 to 3,600·120 + 120 = 432,120 parameters (+107,520). FC1 now accounts for 96.9% of all parameters.
- Total: **446,006** parameters (+31.8% over LeNet-5) for K = 10, and **462,156** for K = 200.
- Compute: same padding makes the conv outputs larger (Conv1 1.84M MACs, Conv2 2.31M MACs), for 4.59M analytic MACs. The profiler reports 4.59M MACs and 9.19 MFLOPs, about 1.28× LeNet-5.

3.3 Full AlexNet-64 (models/alexnet.py)

Textual Description

AlexNet-64 scales the AlexNet design of Krizhevsky et al. (2012) down to 64×64 inputs. The original network used an 11×11 stride-4 first convolution on 224×224 images. Here the first convolution is 5×5 with stride 2, so the small input is not downsampled too early.

Feature extractor. Five convolutional stages with 64, 192, 384, 384 and 256 filters:

- Stages 1–2 use 5×5 kernels.
- Stages 3–5 are three stacked 3×3 convolutions with "same" padding.
- Every convolution is followed by an optional normalization layer and then ReLU, in the order Conv → Norm → ReLU.
- Overlapping max pooling (3×3, stride 2) follows stages 1, 2 and 5.

Classifier. The 256×3×3 output is flattened to 2,304 features and classified by Dropout(0.5) → Linear(2,304, 1,024) → ReLU → Dropout(0.5) → Linear(1,024, 1,024) → ReLU → Linear(1,024, K).

Normalization variants (selected with --norm):

- **none:** nn.Identity in every normalization position.
- **lrn:** LocalResponseNorm(size = 5, $\alpha = 10^{-4}$, $\beta = 0.75$, $k = 2$) after Conv1 and Conv2 only, matching the 2012 paper. Stages 3–5 use Identity.
- **bn:** BatchNorm2d after all five convolutions (64, 192, 384, 384 and 256 channels).

Mathematical Description

$$\text{Conv1: } \lfloor (64 - 5 + 4)/2 \rfloor + 1 = \lfloor 31.5 \rfloor + 1 = 32 \;\rightarrow\; \text{Pool1: } \lfloor (32 - 3)/2 \rfloor + 1 = 15 \;\rightarrow\; \text{Conv2: } (15 - 5 + 4)/1 + 1 = 15 \;\rightarrow\; \text{Pool2: } \lfloor (15 - 3)/2 \rfloor + 1 = 7$$

$$\text{Conv3–5: } (7 - 3 + 2)/1 + 1 = 7 \;\rightarrow\; \text{Pool3: } \lfloor (7 - 3)/2 \rfloor + 1 = 3 \;\rightarrow\; \text{Flatten: } 256 \times 3 \times 3 = 2{,}304$$

Batch Normalization. For channel c, statistics are computed over the mini-batch and both spatial axes (N, H, W). The activations are then normalized and given a learnable scale γ_c and shift β_c:

$$\mu_c = (1/\mathrm{NHW}) \sum_{n,h,w} x_{nchw}, \quad \sigma_c^2 = (1/\mathrm{NHW}) \sum_{n,h,w} (x_{nchw} - \mu_c)^2, \quad y_{nchw} = \gamma_c \cdot (x_{nchw} - \mu_c) / \sqrt{(\sigma_c^2 + \varepsilon)} + \beta_c$$

During evaluation, running averages of μ_c and σ_c^2 (momentum 0.1, $\varepsilon = 10^{-5}$) are used instead of the batch statistics.

Local Response Normalization. LRN divides each activation by the energy of the n = 5 neighbouring channels at the same spatial position. This is a lateral-inhibition scheme with no learnable parameters (PyTorch form):

$$b_{c,h,w} = a_{c,h,w} \,/\, \left(k + (\alpha/n) \sum_{c' = \max(0,\, c - n/2)}^{\min(C - 1,\, c + n/2)} a_{c',h,w}^2 \right)^{\beta}, \quad k = 2,\ n = 5,\ \alpha = 10^{-4},\ \beta = 0.75$$

Dropout (p = 0.5). During training, each input feature to FC1 and FC2 is zeroed with probability p, and the surviving features are rescaled. At inference, dropout is the identity:

$$\tilde{h} = (m \odot h) \,/\, (1 - p), \quad m_i \sim \mathrm{Bernoulli}(1 - p)$$

Layer Dimensions, Receptive Field and Parameter Breakdown

Table 1.3: AlexNet-64 layer specification, BatchNorm variant (input 3×64×64). For the none and LRN variants, delete the "+BN" parameters.

Layer	Operation	Output (C×H×W)	RF (px)	Jump	Params	MACs
Conv1	Conv2d(3→64, 5×5, s=2, p=2) → Norm → ReLU	64×32×32	5	2	4,864 (+128 BN)	4,915,200
Pool1	MaxPool2d(3×3, s=2)	64×15×15	9	4	0	—
Conv2	Conv2d(64→192, 5×5, s=1, p=2) → Norm → ReLU	192×15×15	25	4	307,392 (+384 BN)	69,120,000
Pool2	MaxPool2d(3×3, s=2)	192×7×7	33	8	0	—
Conv3	Conv2d(192→384, 3×3, s=1, p=1) → Norm → ReLU	384×7×7	49	8	663,936 (+768 BN)	32,514,048
Conv4	Conv2d(384→384, 3×3, s=1, p=1) → Norm → ReLU	384×7×7	65	8	1,327,488 (+768 BN)	65,028,096
Conv5	Conv2d(384→256, 3×3, s=1, p=1) → Norm → ReLU	256×7×7	81	8	884,992 (+512 BN)	43,352,064
Pool3	MaxPool2d(3×3, s=2)	256×3×3	97	16	0	—
Flatten	256×3×3 → 2,304	2,304	—	—	0	—
Drop1	Dropout(p = 0.5)	2,304	—	—	0	—
FC1	Linear(2,304→1024) + ReLU	1024	—	—	2,360,320	2,359,296
Drop2	Dropout(p = 0.5)	1,024	—	—	0	—
FC2	Linear(1,024→1024) + ReLU	1024	—	—	1,049,600	1,048,576
FC3 (out)	Linear(1,024→K)	K (10 / 200)	—	—	10,250	10,240
Total	**K = 10 (CIFAR-10)**				**6,611,402**	**218,347,520**

Receptive field: r = 5 → 9 → 25 → 33 → 49 → 65 → 81 → 97 px, with a final jump of 16. Each stacked 3×3 convolution at jump 8 widens the RF by 2·8 = 16 px. This is how three 3×3 layers reach the coverage of a much larger kernel, with fewer parameters and two extra non-linearities. Conv4 already has a 65 px RF, so every unit after Conv4 can integrate information from the whole 64×64 image. Unlike LeNet, the convolutional layers themselves can model global object structure.

Parameter breakdown:

- Convolutional weights: 4,864 + 307,392 + 663,936 + 1,327,488 + 884,992 = 3,188,672 (48.2%). Conv4 is the largest convolution, at 1,327,488 parameters (20.1%).
- Fully connected layers: 2,360,320 + 1,049,600 + 10,250 = 3,420,170 (51.7%). FC1 is the largest single layer, at 2,360,320 parameters (35.7%).
- BatchNorm: 2·(64 + 192 + 384 + 384 + 256) = 2,560 trainable γ/β parameters (0.04%).
- Totals: **6,608,842** for the none and LRN variants (LRN has no parameters) and **6,611,402** for the BN variant. With K = 200, FC3 becomes 1,024·200 + 200 = 205,000, giving **6,806,152** (BN) and 6,803,592 (none/LRN).
- Compute: convolutions account for 214.9M of the 218.3M analytic MACs (98.4%). Conv2 alone uses 69.1M MACs (31.7%). The FC layers hold half of the parameters but use only 1.6% of the

compute. The profiler reports 218.35M MACs / 436.70 MFLOPs (none and LRN) and 218.98M MACs / 437.97 MFLOPs (BN).

Note on handout reference values. The handout quotes about 6.3M parameters and 250–257 MFLOPs for AlexNet-64. Summing the layers specified in Section 3.2 of the handout gives exactly 6,611,402 parameters, which agrees with torchinfo, so our implementation matches the given specification. Our FLOP count also follows directly from the convolution MAC count above, using 2 FLOPs per MAC.

3.4 PyTorch Model Zoo AlexNet (models/zoo.py)

Textual Description

The reference baseline is torchvision.models.alexnet. This is the single-tower AlexNet of Krizhevsky (2014), designed for 224×224 ImageNet images. It is used unchanged, except that the final 4,096 → 1,000 layer is replaced by Linear(4,096, K). It was trained from random initialization without the --pretrained flag, the same way as our own models.

Feature extractor. Five ReLU convolutions with 64, 192, 384, 256 and 256 filters. Conv1 is an aggressive 11×11 stride-4 layer, Conv2 is 5×5, and Conv3–5 are 3×3. There are three overlapping 3×3/stride-2 max pools and **no normalization layers**.

Classifier. AdaptiveAvgPool2d((6, 6)) forces a 256×6×6 = 9,216-dimensional vector for any input size. It feeds Dropout(0.5) → Linear(9,216, 4,096) → ReLU → Dropout(0.5) → Linear(4,096, 4,096) → ReLU → Linear(4,096, K).

Mathematical Description

$$\text{Conv1: } \lfloor (64-11+4)/4 \rfloor + 1 = \lfloor 14.25 \rfloor + 1 = 15 \;\rightarrow\; \text{Pool1: } \lfloor (15-3)/2 \rfloor + 1 = 7 \;\rightarrow\; \text{Conv2: } 7 \;\rightarrow\; \text{Pool2: } \lfloor (7-3)/2 \rfloor + 1 = 3$$

$$\text{Conv3–5: } (3-3+2)/1 + 1 = 3 \;\rightarrow\; \text{Pool3: } \lfloor (3-3)/2 \rfloor + 1 = 1 \;\rightarrow\; \text{AdaptiveAvgPool: } 1\times1 \rightarrow 6\times6 \;\rightarrow\; \text{Flatten: } 256 \times 6 \times 6 = 9{,}216$$

Adaptive average pooling splits the input into a 6×6 grid of bins. Bin i covers rows $\lfloor i \cdot H_{in}/6 \rfloor$ to $\lceil (i+1) \cdot H_{in}/6 \rceil - 1$. With 64×64 inputs, $H_{in} = 1$, so every bin contains the same single pixel and each channel value is copied 36 times. The 9,216-dimensional feature vector therefore holds only 256 distinct numbers. For a 224×224 input, Pool3 would produce a true 6×6 map.

This has two consequences at 64×64 resolution:

- All spatial layout is lost by Pool3.
- FC1's 37.75M weights act on 36 copies of the same 256 features, so they are equivalent to a much smaller 256 → 4,096 layer.

The architecture was designed for larger images, and these consequences explain its weaker results compared with AlexNet-64.

Layer Dimensions, Receptive Field and Parameter Breakdown

Table 1.4: torchvision AlexNet layer specification at 64×64 input.

Layer	Operation	Output (C×H×W)	RF (px)	Jump	Params	MACs
Conv1	Conv2d(3→64, 11×11, s=4, p=2) → ReLU	64×15×15	11	4	23,296	5,227,200
Pool1	MaxPool2d(3×3, s=2)	64×7×7	19	8	0	—
Conv2	Conv2d(64→192, 5×5, s=1, p=2) → ReLU	192×7×7	51	8	307,392	15,052,800
Pool2	MaxPool2d(3×3, s=2)	192×3×3	67	16	0	—
Conv3	Conv2d(192→384, 3×3, s=1, p=1) → ReLU	384×3×3	99	16	663,936	5,971,968
Conv4	Conv2d(384→256, 3×3, s=1, p=1) → ReLU	256×3×3	131	16	884,992	7,962,624
Conv5	Conv2d(256→256, 3×3, s=1, p=1) → ReLU	256×3×3	163	16	590,080	5,308,416
Pool3	MaxPool2d(3×3, s=2)	256×1×1	195	32	0	—
AvgPool	AdaptiveAvgPool2d((6, 6))	256×6×6	—	—	0	—
Flatten	256×6×6 → 9,216	9,216	—	—	0	—
Drop1	Dropout(p = 0.5)	9,216	—	—	0	—
FC1	Linear(9,216→4096) + ReLU	4096	—	—	37,752,832	37,748,736
Drop2	Dropout(p = 0.5)	4,096	—	—	0	—
FC2	Linear(4,096→4096) + ReLU	4096	—	—	16,781,312	16,777,216
FC3 (out)	Linear(4,096→K)	K (10 / 200)	—	—	40,970	40,960
Total	**K = 10 (CIFAR-10)**				**57,044,810**	**94,089,920**

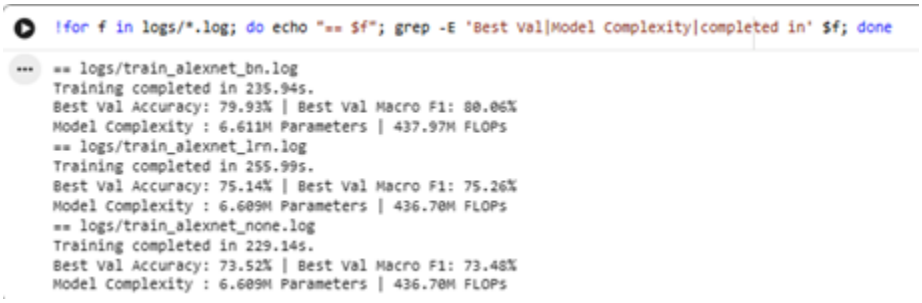
Receptive field: r = 11 → 19 → 51 → 67 → 99 → 131 → 163 → 195 px, with a final jump of 32. The RF exceeds the 64 px image from Conv3 onward, so most of each RF lies in the zero-padding. The stride-4 first layer gives a jump of 4 immediately, so fine detail in a 64×64 image is lost after one layer.

Parameter breakdown:

- Convolutional layers: 23,296 + 307,392 + 663,936 + 884,992 + 590,080 = 2,469,696 (4.3%).
- Fully connected layers: 37,752,832 + 16,781,312 + 40,970 = 54,575,114 (95.7%). FC1 alone holds 66.2% of all parameters.
- Total: **57,044,810** for K = 10. With K = 200, FC3 becomes 4,096·200 + 200 = 819,400, giving **57,823,240**.
- Compute: despite having 8.6× the parameters of AlexNet-64, the zoo model needs only 94.1M MACs (43% of AlexNet-64). Its stride-4 first convolution shrinks the maps to 15×15 immediately, and the convolutions then operate on 7×7 and 3×3 maps instead of 15×15 and 7×7. The profiler reports 94.10M MACs and 188.20 MFLOPs.

Empirical Observations and Questions

Comparing final log lines for each run to see overhead training time.



```
!for f in logs/*.log; do echo "== $f"; grep -E 'Best Val|Model Complexity|completed in' $f; done

== logs/train_alexnet_bn.log
Training completed in 235.94s.
Best Val Accuracy: 79.93% | Best Val Macro F1: 80.06%
Model Complexity : 6.611M Parameters | 437.97M FLOPs
== logs/train_alexnet_lrn.log
Training completed in 255.99s.
Best Val Accuracy: 75.14% | Best Val Macro F1: 75.26%
Model Complexity : 6.609M Parameters | 436.70M FLOPs
== logs/train_alexnet_none.log
Training completed in 229.14s.
Best Val Accuracy: 73.52% | Best Val Macro F1: 73.48%
Model Complexity : 6.609M Parameters | 436.70M FLOPs
```

Performance improvements

Yes, incorporating Batch Normalization yielded improved results in validation accuracy and macro F1 over no normalization and LRN.

Variant	Validation (%)	Marco F1 (%)
No Normalization	73.52	73.48
LRN	75.14	75.26
Batch Normalization	79.93	80.06

Batch Normalization standardizes each channel's activations over the mini-batch, $\hat{x} = (x - \mu_B)/\sqrt{(\sigma_B^2 + \varepsilon)}$, then applies a learnable scale and shift, $y = \gamma\hat{x} + \beta$. Without it, the input distribution of each layer shifts every time the earlier layers' weights update (internal covariate shift), so each layer is chasing a moving target. BN keeps every layer's inputs at a stable mean and variance, which makes the loss surface smoother and its gradients better behaved. Adam can then take larger useful steps at the same learning rate without overshooting. Because BN sits directly before each ReLU, it also keeps pre-activations centred near zero, so fewer units "die" and gradients reach all five convolutional stages. This shows in our runs: BN reached 52.56% validation accuracy after one epoch (vs 46.38% for None and 44.45% for LRN), passed the None variant's final accuracy by epoch 5, and finished at 79.93% / 80.06% F1. LRN gained much less, because with $k = 2$ and $\alpha = 10^{-4}$ it acts almost like a fixed rescaling of the first two stages rather than a true normalization.

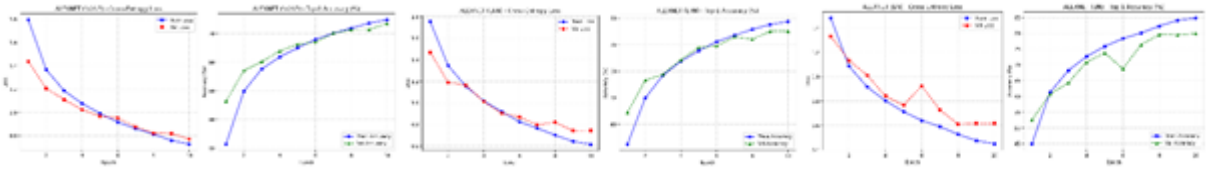
Computational Overhead

Yes for wall clock, no for FLOPS. The addition of normalization to the training does not add very much computation and therefore FLOPs. We can see that in terms on FLOPs only Batch Normalization has a noticeable change, an increase of 1.2 MFLOPs or 0.288% very small.

Variant	*MFLOPs*	*Mean Epoch Time (s)*
No Normalization	*436.7*	*22.84*
LRN	*436.7*	*25.54*
Batch Normalization	*437.97*	*23.53*

In terms of epoch wall time We can see increases in the time for LRN and Batch Normalization with LRN being the worst. Although mean epoch time increases we can look at the number of epochs it takes to reach ~73.5% val accuracy (best achieved with no normalization after 10 epochs).

Training curves for AlexNet runs with different normalizations techniques.

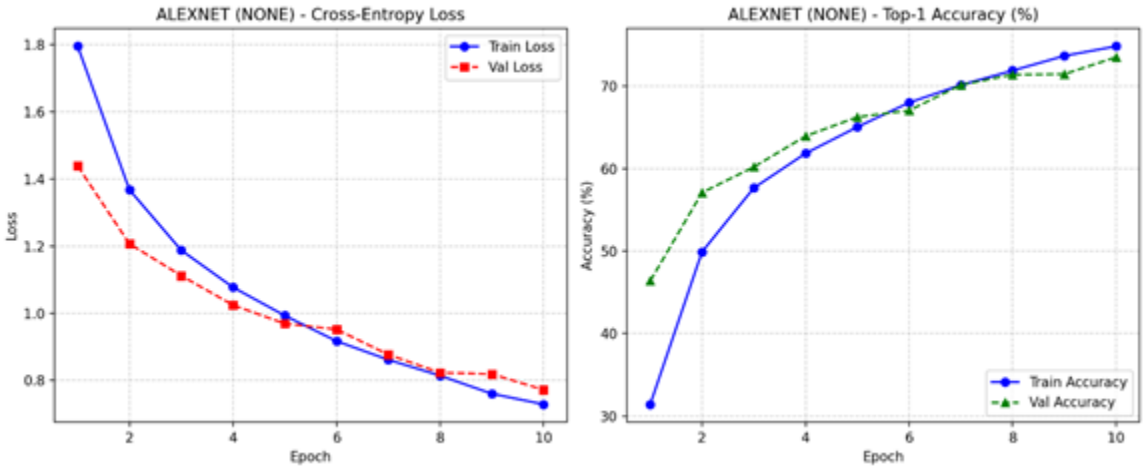


None *LRN* *BN*

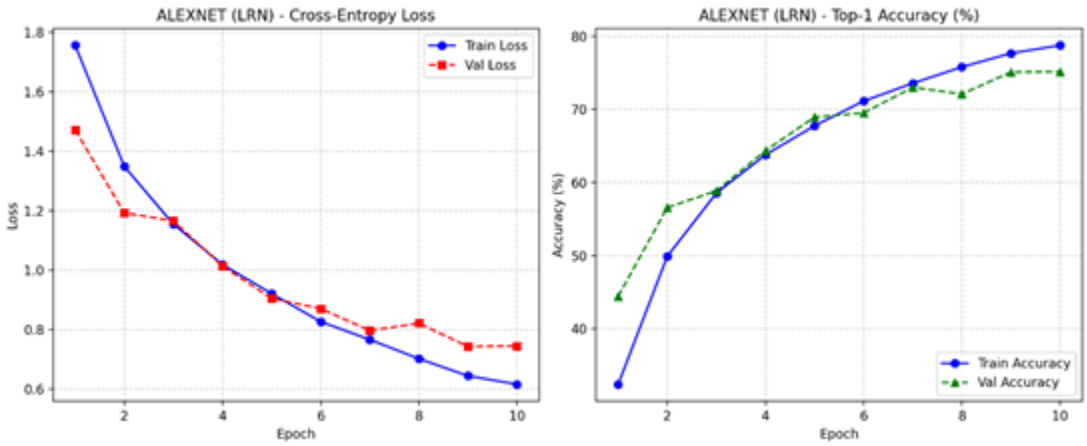
We can see that after around 8 epochs LRN has achieved around 73.5% val accuracy. Using BN we get 73.5% around 5 epochs. We increased mean epoch training by around 8% but halved convergence iterations using BN.

Convergence Stability

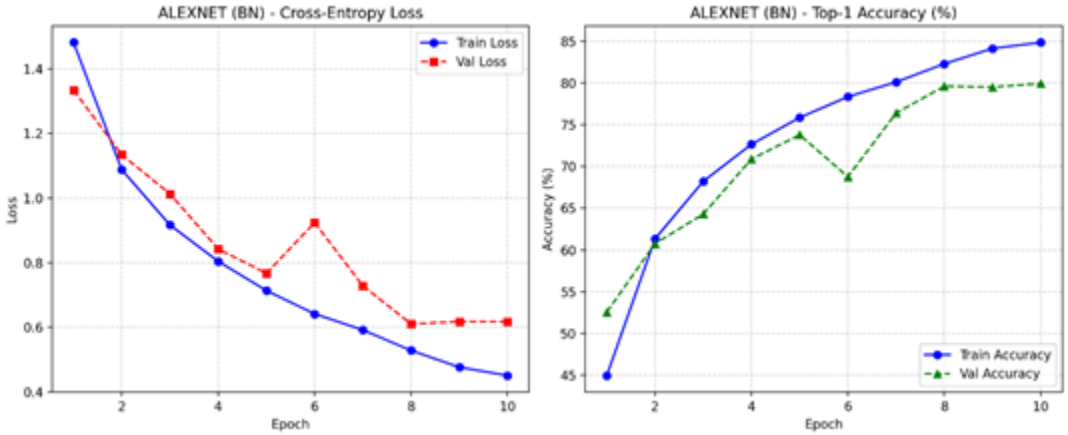
All three variants show smooth and decreasing training loss. Batch normalization shows the best overall gradient progression. It starts of the lowest loss remains the lowest throughout the 10 epochs and ends at 0.45 much lower than 0.63 (LRN) and 0.73(None). All plots have low variance they fall each epoch and have no jumps.



No Normalization

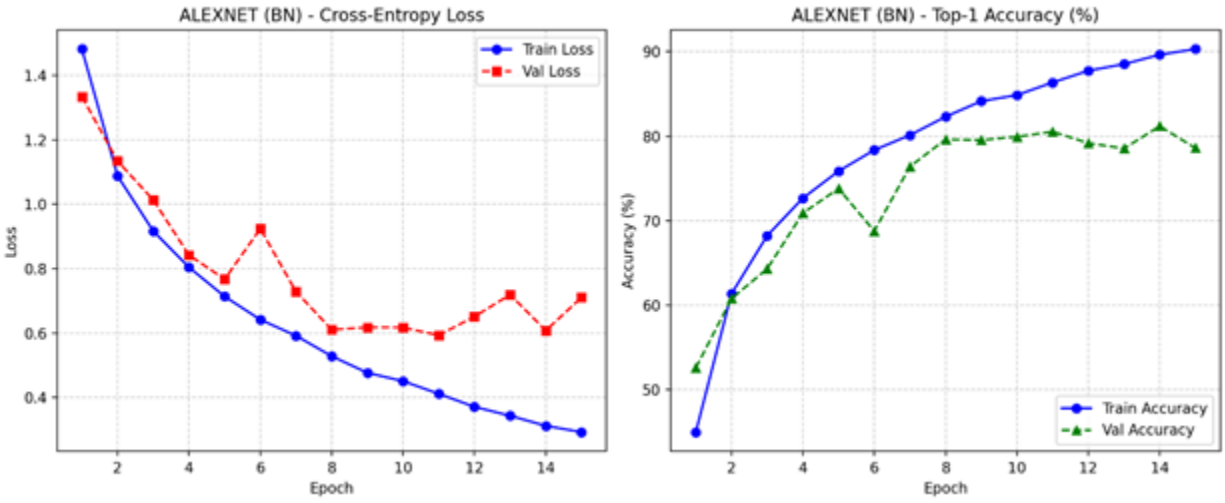


LRN

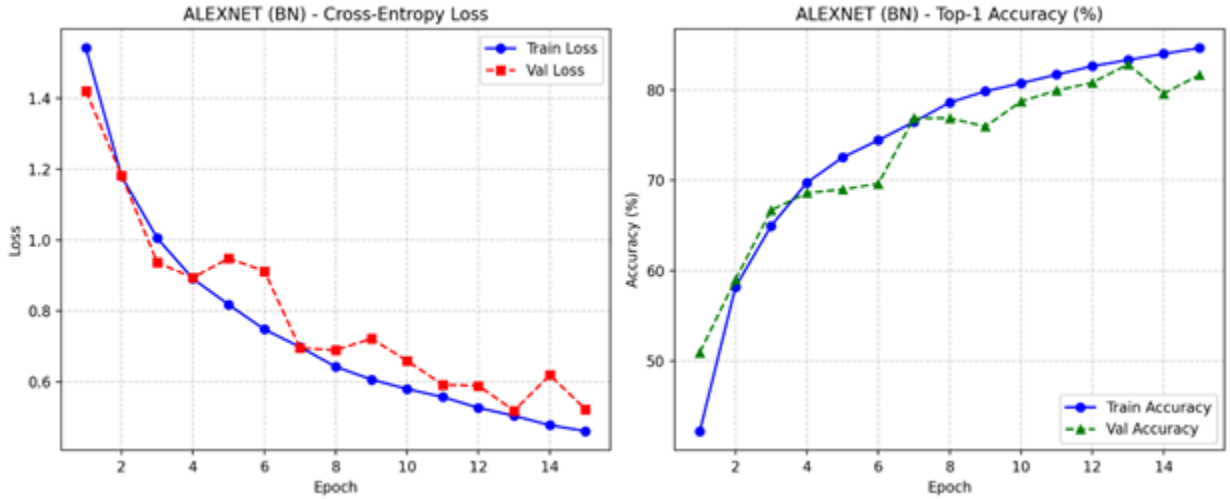


BN

Loss without Augmentation



Loss with augmentation



Measuring the Generalization Gap

Empirical Questions to Address in your report

Did applying PyToch data augmentation reduce the generalization gap Y/N?

Model after 15 Epochs	***Train Acc***	***Val Acc***	***Delta***
No Augmentation	*90.31*	*78.59*	*11.72*
Augmentation	*84.64*	*81.65*	*3*

A: *Yes applying the data augmentation pipeline reduced the gap from 11.72 to 3% after 15 epochs of training.*

Report training accuracy, validation and delta for each of the 2 runs

Unaugmented: Training Accuracy 90.31%, Validation Accuracy on Epoch 15 78.59%.

$$\Delta_{unaugmented} = 0.9031 - 0.7859 = 0.1172 = 11.72\%$$

Augmented: Training Accuracy 84.64%, Validation Accuracy on Epoch 15 81.65%.

$$\Delta_{augmented} = 0.8464 - 0.8165 = 0.0299 \cong 3\%$$

Did data augmentation lead to an overall increase in the final validation accuracy and Macro F_1 yes/no?

A: *Yes from unaugmented 81.18% (validation) 81.08 (macro F1) to augmented 84.64% (validation) 82.90% (macro F1).*

Although the images may come in different orders each epoch the AlexNet will see the same training data each time. But with random crops and flips we are ensuring that AlexNet sees different images throughout the training process, but the images still have the same label. This leads to less likelihood of overfitting and forces AlexNet to learn features not memorize the images. These learned features lead to better results when unknown validation images are used for training.

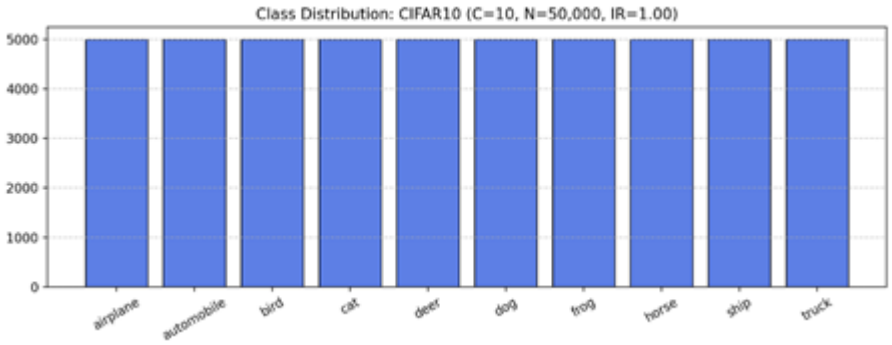
Empirical Questions to Adress

Yes, AlexNet with BN did learn features on Tiny ImageNet above the Random Guess. The best achieved accuracy after 10 epochs was 36.81%. Which is above the random guess of $\frac{1}{200} = 0.5\%$.

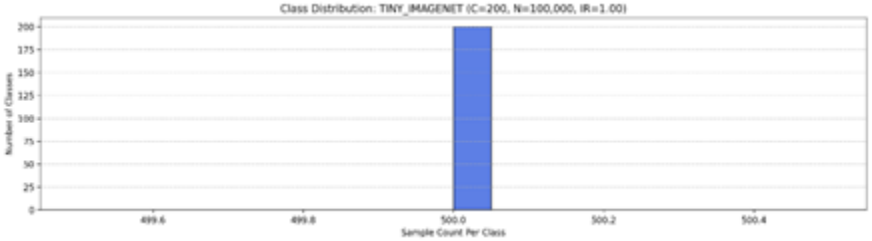
Compared to the CIFAR-10 using AlexNet with BatchNorm we got the following scores.

Model Architecture	***CIFAR-10 VAL Acc (%)***	***CIFAR-10 Val Macro (F_1) (%)***	***Tiny ImageNet Val Acc (%)***	***Tiny ImageNet Val Macro F_1 (%)***
5.AlexNet + BatchNorm	*79.93*	*80.06*	*36.81*	*35.08*

Accuracy and Marco F1 have more than halved when using Tiny ImageNet compared to CIFAR-10. This challenge arises as the images are the same resolution 64x64 and with only 500 training images for 200 classes in Tiny ImageNet vs 5000 for 10 classes in CIFAR1-10 the model may find it difficult to extract distinct features.



CIFAR-10 class image distribution



Tiny ImageNet image class distribution

As well many if the classes in Tiny ImageNet are very similar such as different dog/cat breeds making distinguishing between them very difficult with only 500 training images per class and only 10 epochs.

Overfitting Trajectory: Did the Tiny ImageNet validation loss diverge or plateau earlier in the training schedule compared to CIFAR-10? Explain this behavior using your generated loss curves.

No. On Tiny ImageNet the validation loss neither diverged nor plateaued early. For AlexNet + BN it fell from 4.35 at epoch 1 to 2.61 at epoch 10, and the best validation accuracy (36.81%) came in the final epoch. Training accuracy (30.49%) stayed below validation accuracy throughout, which means the model was still underfitting. On CIFAR-10, the same model's validation loss bottomed out at 0.61 around epoch 8 and then flattened while training loss kept falling. When CIFAR-10 training was extended to 15 epochs without augmentation, validation loss rose from 0.59 to 0.71 while training accuracy reached 90.31%, which is clear overfitting. The difference comes from task difficulty: with 200 classes and only 500 images per class, the network needs far more epochs to fit the Tiny ImageNet training set, so 10 epochs ends before overfitting can begin. CIFAR-10's 10-class task is learned quickly enough for overfitting to appear within the same schedule.

Master Ablation Summary tables

Table 7.3a

Model Architecture	Conv Depth	Normalization and Regularization	Data Augmentation	Trainable Params (M)	FLOPs (MFLOPs)	Mean Epoch Time (s)	Inference Latency (ms)
1.Base LeNet 5	2	None	None	0.338	7.171	21.25	0.91
2.Moderenized LeNet	2	None (ReLU + Overlapping MaxPool)	None	0.446	9.185	21.45	1.38
3.AlexNet (No Norm)	5	Dropout (p=0.5)	None	6.609	436.7	22.84	11.86
4.AlexNet + LRN	5	LRN + Dropout (p=0.5)	None	6.609	436.7	25.54	15.69
5.AlexNet + BatchNorm	5	BatchNorm2d + Dropout (p=0.5)	None	6.611	437.97	23.53	12.11
6.AlexNet + BN + Aug	5	BatchNorm + Dropout (p=0.5)	Pure PyTorch Aug	6.611	437.97	54.08	12.11
7.PyTorch Zoo AlexNet	5	PyTorch Zoo (AdaptiveAvgPool2d)	None	57.045	188.20	28.35	22.91

**All Mean Epoch time run on Colab T4 GPU.*

**"compare_models.py" crashes when using Colab GPU session, so Inference Latency was run on their CPU see terminal output in Empirical Figures, Model Comparisons.*

Table 7.3b

Model Architecture	CIFAR-10 VAL Acc (%)	CIFAR-10 Val Macro (F_1) (%)	Tiny ImageNet Val Acc (%)	Tiny ImageNet Val Macro F_1 (%)
1.Base LeNet 5	54.41	54.30	11.89	9.38
2.Moderenized LeNet	65.36	65.27	21.73	19.18
3.AlexNet (No Norm)	73.52	73.48	18.03	15.13
4.AlexNet + LRN*	75.14	75.26	0.5	0.0
5.AlexNet + BatchNorm	79.93	80.06	36.81	35.08
6.AlexNet + BN + Aug *	82.83	82.9	40.37	38.84
7.PyTorch Zoo AlexNet	63.03	62.80	9.35	6.4

**AlexNet with BranchNorm and Augmentation run for 15 epochs.*

**AlexNet with LRN did not run when using the tiny imageNet, error stays at 0.5 which is 1/200 or a guess.*

AlexNet + LRN failed to train on Tiny ImageNet. Its loss stayed at 5.298 for all 10 epochs, which equals ln(200), the loss of a uniform random guess over 200 classes. Validation accuracy stayed at 1/200 = 0.5% and Macro F1 at 0.0%, meaning the network predicted the same class for every image. The training collapsed in the first epoch and never recovered. Once a network's outputs become constant (for example, when all the ReLUs in a layer output zero), no useful gradient reaches the earlier layers, so the weights stop updating. The same model trained normally on CIFAR-10 (75.14%), so this is a failed run, not evidence that LRN is worse than no normalization. Re-running with a lower learning rate or a different seed would likely fix it.

Extended Ablation Summary tables

Table describing changes made to the model during the extended ablation runs. The baseline is the original AlexNet model with batch normalization (row 5 of ablation tables a) and b)).

Variant	What changes in the model	Params	MFLOPs	Best Val Acc (%)	Macro F1 (%)	Δ Acc (pp)	Train–Val gap (pp)	Epoch (s)
Baseline	NA	6.61 M	437.97	79.93	80.06	NA	4.93	23.53
Seeds 42/43/44	Random seed only	6.61 M	437.97	79.56	79.56	0	NA	NA

No dropout	Dropout 0.5 → 0	6.61 M	437.97	82.38	82.42	2.45	6.34	24.94
Half width	Conv channels ÷2	3.04 M	115.03	80.64	80.82		4.86	22.69
Shallow	Conv stage 4 removed	5.28 M	307.76	79.80	79.77	−0.13	4.74	23.84
AvgPool	MaxPool → AvgPool	6.61 M	438.02	76.66	77.08	−3.27	5.97	25.35

Empirical Figures

Output after running “demo_transforms.py”.

```
================================================================================
ELEC 475: Transform Paradigm & Variable Type Diagnostic
================================================================================

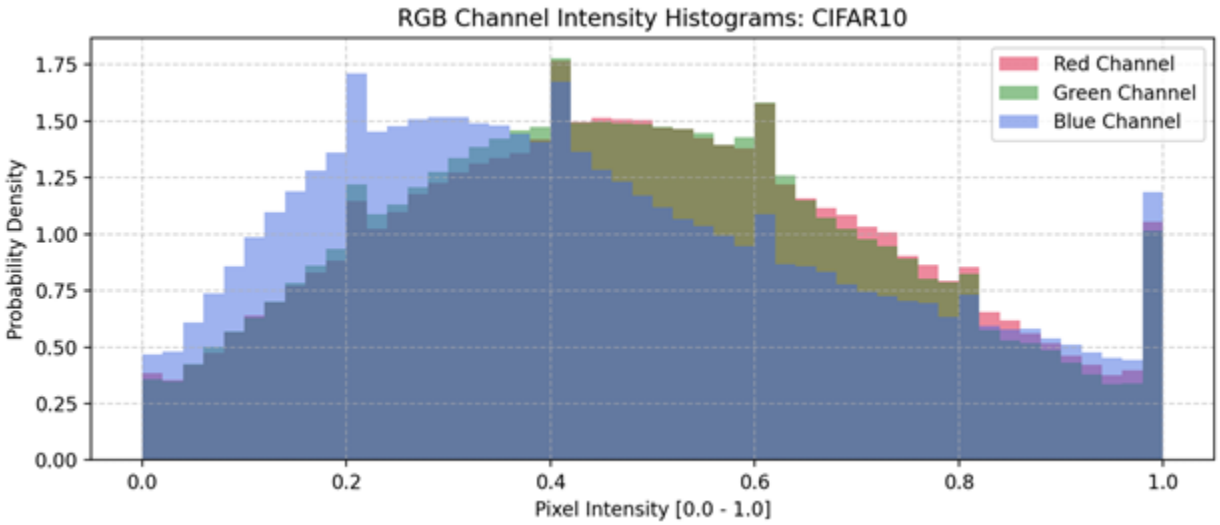
[1] OpenCV (cv2) Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Color Space      : Converted BGR -> RGB
    - Memory Location  : Host System RAM (CPU)

[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)

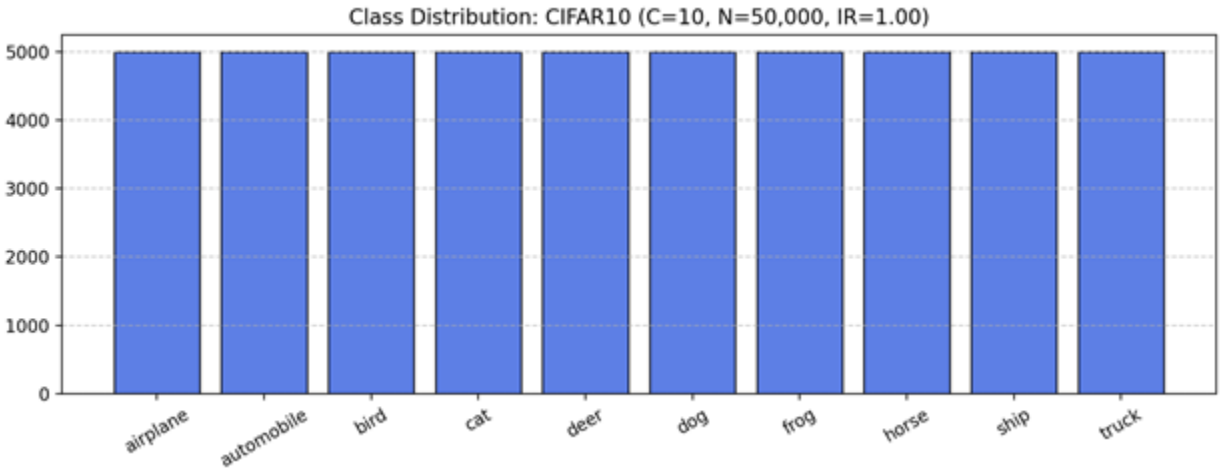
[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height, Width] (Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : cuda:0 (NVIDIA GPU VRAM)

--------------------------------------------------------------------------------
Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
--------------------------------------------------------------------------------
- OpenCV (CPU)            :   11.08 ms  ( 90,289 img/s) + Host-to-Device Copy required
- NumPy (CPU)             :   39.40 ms  ( 25,381 img/s) + Host-to-Device Copy required
- PyTorch (GPU Tensors)   :  264.57 ms  (  3,780 img/s) [ZERO PCIe Transfer Overhead]
================================================================================
```

Eda.py outputs for CIFAR-10



Channel Histogram shows mainly red and green channel colors with lower mean of blue.

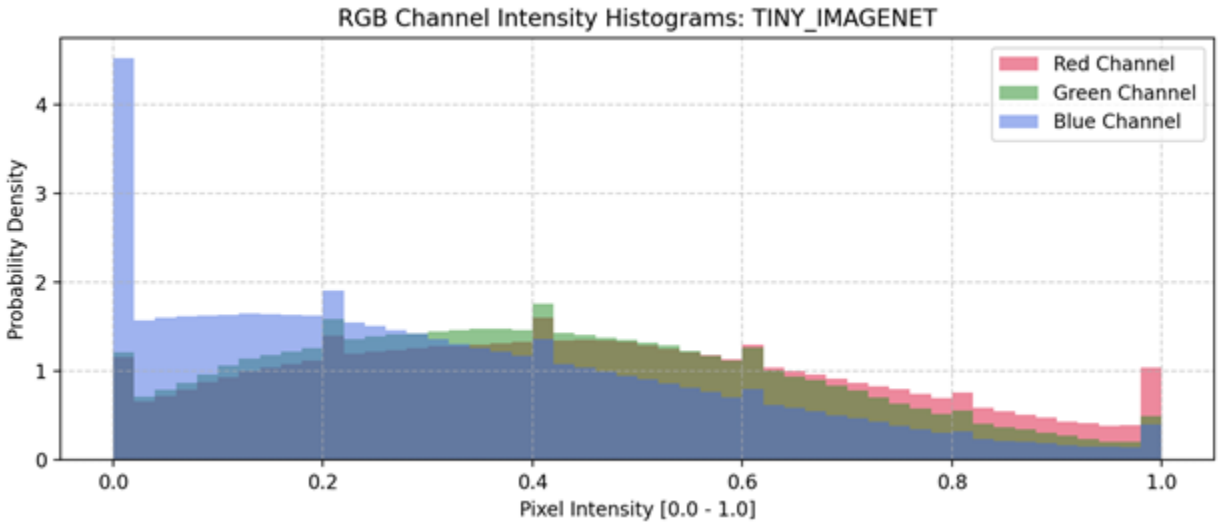


Each class in CIFAR-10 has 5000 training images.

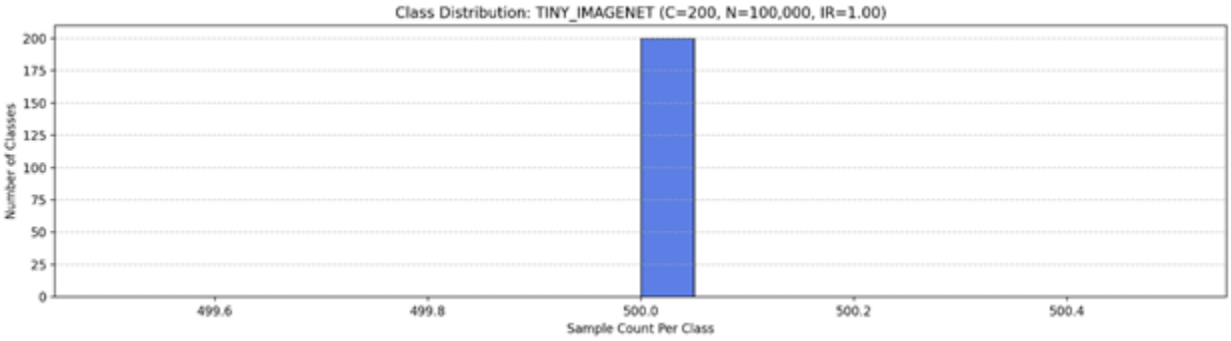


Sample 64x64 grid output of the CIFAR-10 dataset with labels. Much simpler labeling that Tiny ImageNet.

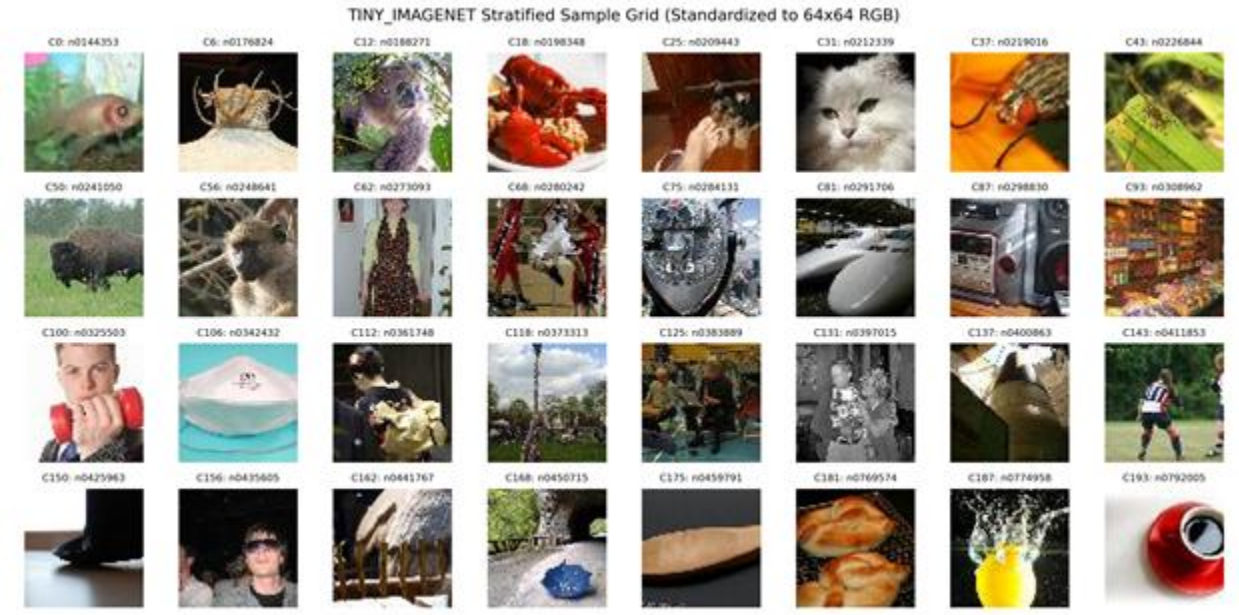
Tiny ImageNet eda.py outputs are below.



Many images with 0 blue channel mainly green and red again.



Fro 200 classes each one has 500 training images.



Output 64x64 image grid showing sample outputs of Tiny ImageNet with associated labels.

Model Comparisons

Outputs from model comparisons used on the Colab CPU since crashes when attempting to use GPU.

```
================================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device        : cpu (CPU)
Dataset       : CIFAR10 (10 classes)
Input Shape   : (1, 3, 64, 64)
================================================================================

Instantiating models for profiling...
Profiling FLOPs, Multiply-Accumulates, and Latency...

| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (ms) | Throughput (img/s) |
| ----------------------------|------------|----------|-----------|--------------|------------------- |
| 1. LeNet-5 (Base)           | 0.338      | 3.59     | 7.17      | 0.91         | 1103.6             |
| 2. LeNet-Modern (ReLU+Pool) | 0.446      | 4.59     | 9.19      | 1.38         | 724.5              |
| 3. AlexNet-64 (None)        | 6.609      | 218.35   | 436.70    | 11.86        | 84.3               |
| 4. AlexNet-64 (LRN)         | 6.609      | 218.35   | 436.70    | 15.69        | 63.8               |
| 5. AlexNet-64 (BatchNorm)   | 6.611      | 218.98   | 437.97    | 12.11        | 82.6               |
| 6. Torchvision Zoo AlexNet  | 57.045     | 94.10    | 188.20    | 22.91        | 43.6               |

================================================================================
Academic Notes on Complexity Metrics:
1. FLOPs vs MACs: 1 Multiply-Accumulate (MAC) corresponds to ~2 Floating Point Operations (FLOPs).
2. Memory vs Compute: Dense Linear layers contribute the majority of parameters, but Conv layers dominate FLOPs.
3. Reference: He et al. (CVPR 2016), Howard et al. (MobileNets 2017).
================================================================================
```

CIFAR-10 outputs

```
================================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device        : cpu (CPU)
Dataset       : TINY_IMAGENET (200 classes)
Input Shape   : (1, 3, 64, 64)
================================================================================

Instantiating models for profiling...
Profiling FLOPs, Multiply-Accumulates, and Latency...

| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (ms) | Throughput (img/s) |
| ----------------------------|------------|----------|-----------|--------------|------------------- |
| 1. LeNet-5 (Base)           | 0.355      | 3.60     | 7.20      | 0.87         | 1145.8             |
| 2. LeNet-Modern (ReLU+Pool) | 0.462      | 4.61     | 9.22      | 1.39         | 721.0              |
| 3. AlexNet-64 (None)        | 6.804      | 218.54   | 437.08    | 11.54        | 86.6               |
| 4. AlexNet-64 (LRN)         | 6.804      | 218.54   | 437.08    | 15.21        | 65.8               |
| 5. AlexNet-64 (BatchNorm)   | 6.806      | 219.18   | 438.36    | 11.86        | 84.3               |
| 6. Torchvision Zoo AlexNet  | 57.823     | 94.88    | 189.76    | 22.73        | 44.0               |

================================================================================
Academic Notes on Complexity Metrics:
1. FLOPs vs MACs: 1 Multiply-Accumulate (MAC) corresponds to ~2 Floating Point Operations (FLOPs).
2. Memory vs Compute: Dense Linear layers contribute the majority of parameters, but Conv layers dominate FLOPs.
3. Reference: He et al. (CVPR 2016), Howard et al. (MobileNets 2017).
================================================================================
```

Tiny ImageNet outputs.

Technical Analysis & Discussion

Base LeNet-5 vs. Modernized LeNet. Replacing Tanh with ReLU and average pooling with overlapping max pooling raised CIFAR-10 validation accuracy from 54.41% to 65.36% (+10.95 points) and Tiny ImageNet accuracy from 11.89% to 21.73%, using the same number of filters. ReLU does not saturate for positive inputs, so gradients pass back without shrinking and training converges faster. Max pooling keeps the strongest edge responses instead of averaging them away. The cost was small: FLOPs rose from 7.17M to 9.19M, yet epoch time on the T4 barely changed (21.25 s → 21.45 s), because small models do not saturate the GPU. Only CPU latency rose noticeably (0.91 ms → 1.38 ms).

Modernized LeNet vs. AlexNet (no normalization). Increasing depth from 2 to 5 convolutional stages, with 64–384 channels, raised CIFAR-10 accuracy to 73.52% (+8.16 points). The deeper network builds a hierarchy of features, and its receptive field grows from 19 px to 97 px, so the final layers can see the whole object. Despite about 47× more FLOPs, T4 epoch time rose only 7% (22.84 s), while CPU latency rose about 8.6× (11.86 ms). On Tiny ImageNet, however, the unnormalized AlexNet (18.03%) scored below Modernized LeNet (21.73%): without normalization, the deeper network is harder to optimize on a 200-class task in 10 epochs.

Normalization (None vs. LRN vs. BatchNorm). On CIFAR-10, BatchNorm gave the best result (79.93% accuracy / 80.06% Macro F1), ahead of LRN (75.14%) and no normalization (73.52%). On Tiny ImageNet, BN roughly doubled the unnormalized result (36.81% vs 18.03%). BN keeps every layer's inputs at a stable mean and variance, which smooths optimization and speeds convergence. LRN acts almost like a fixed rescaling and gives little of this benefit. BN was also cheap, adding about 0.3% FLOPs and 3% epoch time. LRN was the slowest variant (25.54 s per epoch, 15.69 ms latency), even though the profiler counts it as zero FLOPs, because it is a memory-bound cross-channel operation.

Data augmentation. Over 15 epochs, augmentation cut the CIFAR-10 generalization gap from 11.72 to 2.99 points and raised the best validation accuracy from 81.18% to 82.83% (Macro F1 81.08% → 82.90%). Random crops, flips and colour jitter stop the network from memorizing exact training pixels, so it learns features that transfer to unseen images. The cost is training time only: epoch time roughly doubled (54.08 s), because the transforms run image by image on the CPU. Parameters, FLOPs and inference latency are unchanged, since augmentation is not used at test time.

CIFAR-10 vs. Tiny ImageNet. Every model scored far lower on Tiny ImageNet; the best result fell from 82.83% to 40.37%. The task has 20× more classes, only 500 training images per class, and many visually similar categories. Relative to chance, though, the result is strong: 40.37% is about 80× the 0.5% random baseline, whereas 82.83% is about 8× CIFAR-10's 10% baseline. Within 10 epochs the Tiny ImageNet models were still underfitting rather than overfitting. The AlexNet + LRN run failed to train on this dataset (0.5%), as explained under Table 7.3b.

PyTorch Zoo AlexNet. The torchvision AlexNet has the most parameters (57.0M, 8.6× our AlexNet-64), yet it scored only 63.03% on CIFAR-10 and 9.35% on Tiny ImageNet, below even Modernized LeNet. It was designed for 224×224 images. On 64×64 inputs, its 11×11 stride-4 first convolution

discards fine detail immediately, and the feature map shrinks to 1×1 before the adaptive pool simply copies it 36 times, so most of its huge fully connected layers learn from repeated values. It also has no normalization. It had the highest CPU latency (22.91 ms) despite fewer FLOPs than AlexNet-64 (188 vs 437 MFLOPs), because reading its large weight matrices is memory-bound. Matching the architecture to the input size mattered more than parameter count.